Cypress E2E Allure Framework — Code Review Report

Project: AZANIR/cypressAllure
Reviewer: Konstantin Kovalenko
Date: 2026-07-06
Scope: Cypress UI Automation Framework Review (Page Objects, Tests, Config, Reporting, CI/CD)

---

1. Project Overview

This project is a Cypress-based end-to-end automation framework with Allure reporting integration. It demonstrates basic UI test automation using the Page Object Model (POM), fixture-based test data, and Allure reporting.

The framework targets a demo e-commerce application and includes login functionality tests, Page Object implementations, and CI/CD configuration.

---

2. Overall Assessment

The project demonstrates a solid understanding of Cypress fundamentals, including:

- Page Object Model usage
- Test data separation using fixtures
- Cypress configuration setup
- Allure reporting integration
- Basic Mocha test structure

However, several issues were identified related to maintainability, consistency, test design quality, and CI/CD execution reliability. The framework is closer to a learning or sample project than a production-ready solution.

---

3. Framework Architecture Overview

The project follows a layered automation structure:

- **Test Layer:** Cypress test specifications (cypress/e2e/tests)
- **Page Object Layer:** UI abstraction using POM (cypress/pages)
- **Test Data Layer:** JSON fixtures (cypress/fixtures)
- **Configuration Layer:** Cypress settings (config/config.config.ts)
- **Support Layer:** Global commands and plugins (cypress/support)

- **Reporting Layer:** Allure integration and report generation
- **CI/CD Layer:** GitHub Actions workflow automation

This separation improves readability and maintainability of the framework.

---

4. Findings and Recommendations

4.1 README Documentation

Positive aspects

- Provides basic project overview
- Includes setup steps and execution commands
- Shows project structure and reporting examples

Finding 1 — Missing project context and application details

Severity: Medium

Observation:
README does not define application URL, project scope, or execution requirements.

Recommendation:
Add application description, URL, and environment prerequisites (Node.js, Cypress versions).

Finding 2 — Incomplete npm scripts documentation

Severity: Low

Observation:
Not all *npm* scripts from *package.json* are documented.

Recommendation:
Add a dedicated “Scripts” section in *README* explaining each command.

Finding 3 — Non-searchable project structure

Severity: Low

Observation:
Project structure is shown as an image.

Recommendation:
Replace image with Markdown tree format for better readability.

Finding 4 — Missing technology stack section

Severity: Low

Observation:
No explicit technology stack is defined.

Recommendation:
Add a “Tech Stack” section (Cypress, TypeScript, Node.js, Allure).

4.2 package.json

Positive aspects

- Well-organized scripts
- Proper dependency separation
- Complete repository metadata

Finding 1 — Cross-platform script issue

Severity: Medium

Observation:
Unix-specific cleanup command may fail on Windows.

Recommendation:
Replace with cross-platform tool such as *rimraf*.

Finding 2 — Description formatting inconsistency

Severity: Very Low

Observation:
Escaped quotes in description field reduce readability.

Recommendation:
Simplify and clean description formatting.

4.3 Cypress Configuration

Positive aspects

- Proper use of *defineConfig*
- Centralized timeout configuration
- Base URL configured
- Screenshots and videos enabled

Finding 1 — Legacy comments present

Severity: Low

Observation:
Old migration comments remain in config.

Recommendation:
Remove outdated comments for clarity.

Finding 2 — HTTP base URL usage

Severity: Low

Observation:
Application uses HTTP instead of HTTPS.

Recommendation:
Use HTTPS if supported.

Finding 3 — Deprecated plugin structure

Severity: Medium

Observation:
The project uses the deprecated *cypress/plugins* structure instead of initializing plugins directly in *setupNodeEvents().*

Recommendation:
Move plugin initialization into *setupNodeEvents()* and remove the legacy plugin file.

Finding 4 — Allure path mismatch

Severity: High

Observation:
The Allure results path in *config.config.ts* does not match the path used in the report generation script.

Recommendation:
Use the same Allure results directory in both configuration and scripts.

4.4 Support Folder

Positive aspects

- Proper global support structure
- Allure plugin correctly initialized
- Commands file prepared for extension

4.5 Page Objects

Positive aspects

- Correct Page Object Model implementation
- Centralized locators
- Clear method structure

Finding 1 — Assertions inside Page Objects

Severity: Low

Observation:
Validation logic exists inside Page Objects.

Recommendation:
Consider to move assertions to test layer for better separation of concerns.

Finding 2 — Weak method naming

Severity: Low

Observation:
Method names do not always reflect behavior clearly.

Recommendation:
Use descriptive names like *openHomePage()* instead of *launchApplication()*.

Finding 3 — Tight selector coupling

Severity: Low

Observation:
Some selectors depend on UI structure.

Recommendation:
Prefer stable selectors (data attributes or IDs).

Finding 4 — Redundant TypeScript reference

Severity: Very Low

Observation:
Unnecessary triple-slash directive is used.

Recommendation:
Remove redundant TypeScript reference.

4.6 Fixtures

Positive aspects

- Clean separation of test data
- Well-structured valid/invalid scenarios
- Easy to maintain

Finding 1 — Naming inconsistency

Severity: Very Low

Observation:
Mixed naming styles in JSON keys.

Recommendation:
Use consistent naming convention across fixtures.

4.7 Test Files

Positive aspects

- Use of fixtures
- Page Object usage
- Clear test structure

Finding 1 — Placeholder test present

Severity: Medium

Observation:
One test does not validate real application behavior.

Recommendation:
Replace with meaningful test or remove.

Finding 2 — Unused imports

Severity: Low

Observation:
Some imports are unused.

Recommendation:
Remove unused imports.

Finding 3 — Weak test naming

Severity: Low

Observation:
Test names do not fully describe workflows.

Recommendation:
Use descriptive scenario-based naming.

Finding 4 — Mixed data strategy

Severity: Low

Observation:
Hardcoded and fixture data are mixed.

Recommendation:
Use a single consistent data-driven approach.

Finding 5 — Application incompatibility

Severity: High

Observation:
The configured application URL no longer contains the user interface expected by the framework.

Recommendation:
Update the framework to target a currently available test application.

Finding 6 — Incorrect Mocha context usage

Severity: High

Observation:
beforeEach() uses an arrow function while sharing data through *this*, which breaks the Mocha context.

Recommendation:
Use a standard function or replace *this* with Cypress aliases.

Finding 7 — Typo in assertion

Severity: Medium

Observation:
One expected error message contains a spelling mistake, causing the assertion to fail.

Recommendation:
Correct the expected assertion text.

4.8 Allure Reporting / Docs

Positive aspects

- Full Allure integration
- Attachments and history supported
- Execution logs preserved

Finding 1 — Committed generated reports

Severity: Low

Observation:
Generated report artifacts are stored in repository.

Recommendation:
Exclude generated reports from version control.

4.9 Repository Commit History

Positive aspects

- Full development history present

Finding 1 — Poor commit message quality

Severity: Low / Informational

Observation:
Commit history contains non-descriptive messages (e.g., “up”, “123”, “fhj”).

Recommendation:
Adopt conventional commit format (feat, fix, refactor, docs).

4.10 CI/CD Pipeline

Positive aspects

- Full CI pipeline defined
- Includes test execution, reporting, deployment
- Allure integration present
- Artifacts uploaded

Finding 1 — No workflow execution history

Severity: Medium

Observation:
GitHub Actions workflow exists but has no runs.

Recommendation:
Verify triggers and ensure CI is active on correct branch.

Finding 2 — Outdated Node.js version

Severity: Low / Medium

Observation:
Node.js 14 is used in CI.

Recommendation:
Upgrade to Node.js 18 or 20 LTS.

Finding 3 — Legacy GitHub Pages action

Severity: Low

Observation:
Old action version used for deployment.

Recommendation:
Upgrade to latest stable version.

Finding 4 — Incorrect secret reference

Severity: High

Observation:
The GitHub Actions secret is referenced using invalid syntax, preventing the deployment token from being resolved.

Recommendation:
Use the standard syntax: *${{ secrets.ACCESS_TOKEN }}*.

Finding 6 — Workflow recursion risk

Severity: High

Observation:
The workflow deploys reports to the same branch that triggers the pipeline, creating a risk of repeated executions.

Recommendation:
Deploy reports to a separate branch such as *gh-pages* or exclude deployment commits from triggering the workflow.

5. Data Flow Explanation

The framework follows a structured execution flow:

1. Test execution triggered locally or via CI
2. Fixture data loaded into tests
3. Test layer executes scenarios
4. Page Objects perform UI interactions
5. Application responds via browser automation
6. Assertions validate results
7. Allure captures execution data
8. HTML report is generated and published

---

6. Overall Framework Maturity Conclusion

The framework demonstrates solid foundational knowledge of Cypress automation, including Page Object Model usage, fixture-based data handling, and Allure reporting integration.

However, the current implementation is closer to a **learning or sample project** rather than a production-ready automation framework due to inconsistent test design, placeholder tests, inactive CI execution, and incomplete documentation.

With improvements in consistency, CI activation, and documentation quality, the framework could be elevated to a production-grade automation solution.